Universidad Nacional de General Sarmiento

Trabajo práctico
Introducción a la programación

Alcaraz ian
D.N.I : 43.862.088
e-mail : ianalcaraz20@hotmail.com

Peralta lucio
D.N.I : 46.584.131
e-mail :lucioperalta014@gmail.com

## INTRODUCCION:

En este trabajo práctico se trata de hacer (con una página ya empezada por los profesores) distintas funciones y agregar código para su funcionamiento como por ejemplo para que se muestran las tarjetas de los personajes, para que cada tarjeta tenga su información, etc…

A continuación se mostrará cuáles fueron las funciones fundamentales para que esta página funcione.

1) View.py “home(request)”

```python
def home(request):
    """
    Vista principal que muestra la galería de personajes de Los Simpsons.

    Esta función debe obtener el listado de imágenes desde la capa de servicios
    y también el listado de favoritos del usuario, para luego enviarlo al template 'home.html'.
    Recordar que los listados deben pasarse en el contexto con las claves 'images' y 'favourite_list'.
    """
    images = services.getAllImages()
    fav_object = services.getAllFavourites(request)
    favourite_list = [f.name for f in fav_object] # creamos una lista de nombres



    return render(request, 'home.html', { 'images': images, 'favourite_list': favourite_list })
```

simplemente llamamos a tres listas, la primera para guardar las imágenes desde service, y la segunda para almacenar los favoritos, la tercera lista es para obtener los nombres de los favoritos, para poder evitar duplicados

search(request):

```python
def search(request):
    """
    Busca personajes por nombre.

    Se debe implementar la búsqueda de personajes según el nombre ingresado.
    Se debe obtener el parámetro 'query' desde el POST, filtrar las imágenes según el nombre
    y renderizar 'home.html' con los resultados. Si no se ingresa nada, redirigir a 'home'.
    """
    query = request.POST.get('query', '') # busca el query

    if query !='': # si el query es distinto que vacio, entra en el if
        images = services.filterByCharacter(query) # filtra el por el nombre

        favourite_list = services.getAllFavourites(request) if request.user.is_authenticated else [] # 

        return render(request, 'home.html', { 'images': images, 'favourite_list': favourite_list }) # d
    else:
        return redirect('home') # si no escribio nada, lo devuelve al home


    pass
```

Esta función toma los datos que el usuario escribe y los procesa para retornar la búsqueda, si existe el personaje va a renderizar, pero si no existe o no escribió nada lo devuelve al home

Filter_by_status(request):

```python
def filter_by_status(request):
    """
    Filtra personajes por su estado (Alive/Deceased).

    Se debe implementar el filtrado de personajes según su estado.
    Se debe obtener el parámetro 'status' desde el POST, filtrar las imágenes según ese estado
    y renderizar 'home.html' con los resultados. Si no hay estado, redirigir a 'home'.
    """

    status = request.POST.get('status', '') # busca el estado
    if status != '': # si el estado es distinto a vacio entra en el condicional
        images = services.filterByStatus(status)# filtra por el estado
        favourite_list = services.getAllFavourites(request) if request.user.is_authenticated else [] #

        return render(request, 'home.html', { 'images': images, 'favourite_list': favourite_list }) #
    else:
        return redirect('home') # si esta vacio te devuelve a home
    pass
```

Similar a la anterior pero en este caso obtenemos el status con el POST, lo filtra y retorna la búsqueda, en estas funciones utilizamos funciones que traemos desde services, que explicaremos más adelante

registrar_usuario(request)

```python
def registrar_usuario(request):
    if request.method == 'POST':
        form = registroUsuarioForm(request.POST) # llenamos el formulario con los datos
        if form.is_valid():
            user = form.save()# se crea el usuario en la db
            username = form.cleaned_data.get('username') #obtengo los datos para el mensaje
            password = form.cleaned_data.get('password1') #
            email_destinatario = form.cleaned_data.get('email')
            subject = 'Bienvenido al TP IP'
            message = f'Hola{user.first_name},\n\nTu registro fue exitoso.\nUsuario {username}
            \nContrasenia: {password}'
            #email_from = settings.EMAIL_HOST_USER
            #recipient_list =[email_destinatario]
            send_mail(subject, message, settings.EMAIL_HOST_USER, [email_destinatario])
            return redirect('login')
    else:
        form=registroUsuarioForm()
    return render(request, 'registration/register.html', {'form':form})
```

Esta función se encarga de registrar usuarios, django se encarga de mostrar el formulario, fijarse que los datos no sean duplicados, y si es así crea un usuario nuevo en la base de datos. Después manda un mail con un mensaje al usuario registrado con su usuario y contraseña. Esta es una de las funciones que mas nos costo, pero con la ayuda del repositorio de Mohammed Jassim Anikkada (https://github.com/akjasim/cb_django-sending-emails) pudimos lograr que funcionara.

getAllFavouritesByUser(request)

```python
@login_required
def getAllFavouritesByUser(request):
    """
    Obtiene todos los favoritos del usuario autenticado.
    """

    favourite_list = services.getAllFavourites(request) # obtenemos los favoritos

    return render(request, 'favourites.html', {'favourite_list': favourite_list}) # los retornamos
    pass
```

obtiene una lista con todos los favoritos del usuario logueado, y lo retorna

saveFavourite(request)

```python
@login_required
def saveFavourite(request):
    """
    Guarda un personaje como favorito.
    """

    if request.method == 'POST':
        services.saveFavourite(request) # sevices guarda el fav
    return redirect('home')    #retornamos home
    pass
```

Función sencilla que guarda los personajes como favorito

deleteFavourite(request)

```python
def deleteFavourite(request):
    """
    Elimina un favorito del usuario.
    """

    if request.method == 'POST':
        services.deleteFavourite(request)
    return redirect('favoritos')
    pass
```

Función prácticamente igual que la anterior solo que en elimina el favorito

Services.py (Capa de servicio)

```python
def getAllImages():
    """
    Obtiene todas las imágenes de personajes desde la API y las convierte en objetos Card.

    Esta función debe obtener los datos desde transport, transformarlos en Cards usando
    translator y retornar una lista de objetos Card.
    """
    json_collection = transport.getAllImages() # guardo la lista que devuelve transport
    cards = [] # creo una lista para almacenar las cards

    for personaje in json_collection: #recorro json collection
        objeto_card = translator.fromRequestIntoCard(personaje) #funcion para convertir inf
        una card
        cards.append(objeto_card) # le agrego objeto card a la lista de cards
    return cards
```

Esta función obtiene los datos del transport , los convierte en cards y devuelve la lista con las cards, es la primer función a desarrollar en el tp, sencilla ya que no hay que utilizar django, es prácticamente python puro

```python
def filterByCharacter(name):
    """
    Filtra las cards de personajes según el nombre proporcionado.

    Se debe filtrar los personajes cuyo nombre contenga el parámetro recibido. Retorna una lista de Cards
    filtradas.
    """
    all_cards = getAllImages() # llama a los datos de personajes en cards
    cards_filtrada = [] # lista vacia para almacenar los resultados de la busqueda
    for personaje in all_cards: # itearmos en las cards de personajes
        if name.lower() in personaje.name.lower(): # comparamos la iteracion con la busqueda, pasamos a
        minuscula todo para evitar problemas
            cards_filtrada.append(personaje) # si hay encuentra la correcta, se agrega el resultado a la
            nueva lista
    return cards_filtrada # retorna la lista
```

Función que filtra las cards según el nombre proporcionado, similar a la anterior. Al ser python puro fue más  fácil de desarrollar. Esta funcion la utilizamos en la funcion search() en [views.py]

```python
def filterByStatus(status_name):
    """
    Filtra las cards de personajes según su estado (Alive/Deceased).

    Se deben filtrar los personajes que tengan el estado igual al parámetro 'status_name'. Retorna una
    lista de Cards filtradas.
    """

    all_cards = getAllImages() # llamamos a los datos del pj en cards
    cards_filtrada =[] # creo una lista para almacenar
    for personaje in all_cards: # recorro la lista
        if personaje.status.lower() == status_name.lower(): # comparo si es el mismo estado Alive/Deceas
            cards_filtrada.append(personaje) # si es asi, lo agrego a la lista
    return cards_filtrada         # retorno la lista con pj del mismo estado
    pass
```

Función idéntica a la anterior con la diferencia que en vez de buscar por nombre, busca por estado en este caso es Alive/Deceased. Se utiliza en filter_by_status() en [views.py]

```python
# añadir favoritos (usado desde el template 'home.html')
def saveFavourite(request):
    """
    Guarda un favorito en la base de datos.

    Se deben convertir los datos del request en una Card usando el translator,
    asignarle el usuario actual, y guardarla en el repositorio.
    """

    fav = translator.fromRequestIntoCard(request.POST) #convertimos los datos del POST a una card
    fav.user = request.user # se la asignamos al usuario
    return repositories.saveFavourite(fav)
    pass
```

convierte los datos del request en un card y los asigna al usuario que tiene la sesion iniciada y lo guarda en repositories

```python
def getAllFavourites(request):
    """
    Obtiene todos los favoritos del usuario autenticado.

    Si el usuario está autenticado, se deben obtener sus favoritos desde el repositorio,
    transformarlos en Cards usando translator y retornar la lista. Si no está autenticado, se retorna una
    lista vacía.
    """
    if not  request.user.is_authenticated:
        return []
    user_favs = repositories.getAllFavourites(request.user) # buscamos los favoritos en el repo

    favourite_list = [] # lista vacia para almacenar las cards
    for fav in user_favs: # recorremos los favs del usuario
        favourite_list.append(translator.fromRepositoryIntoCard(fav)) # agregamos la card a la lista
        print(user_favs)
    return favourite_list #retornamos la lista con las cards

    pass
```

Función que si estás autenticado retorna una lista  con card favoritos obteniéndose desde repositorios, si no está autenticado retorna una lista vacía

```python
def deleteFavourite(request):
    """
    Elimina un favorito de la base de datos.

    Se debe obtener el ID del favorito desde el POST y eliminarlo desde el repositorio.
    """

    favId = request.POST.get('id') # obtenemos el id
    print(f"Intentando borrar favorito con ID: {favId}") # Esto saldrá en tu terminal
    return repositories.deleteFavourite(favId)
    #return repositories.deleteFavourite(favId) # se elimina desde el repo

    pass
```

Función que elimina el favorito desde la base de datos, obtiene la información desde el POST y lo elimina en repositories

```
<!--Cambio de Borde en base a su estado de vida.-->
<div class="card h-100 shadow-sm
    {% if img.status == 'Alive' %}
        border-success
    {% elif img.status == 'Deceased' %}
        border-secondary
    {% else %}
        border-warning
    {% endif %}
">
```

Utilizamos django para cambiar los bordes en base al estado del personaje, con ayuda del bootstrap

```
<div class="alert alert-warning p-2 text-center">
    <!-- Mostrar una frase aleatoria del personaje o una cadena vacía si no tiene frases -->
    {% if img.phrases %}
        "{{ img.phrases|random }}"
    {% else %}
        ""
    {% endif %}
</div>
```

devuelta con ayuda de django se realizó un if para que la card muestre una frase aleatoria, en caso de que no tenga frase se agregan unas comillas

# Dificultades

Una de nuestras primeras dificultades fue la utilización de los repositorios de github, tuvimos muchos errores al principio, teniendo que eliminar ese repositorio para comenzar de 0. Los problemas con github lo logramos solucionar viendo todos los videos explicativos de git que proporcionaron los docentes, y con práctica.
Al realizar las primeras funciones no tuvimos ningún tipo de inconveniente ya que era todo lo aprendido en clase, pero empezamos a tener muchos errores cuando tuvimos que hacer los extras y utilizar mucho DJANGO, no entendiamos como utilizar bien las funciones que disponíamos de django, lo pudimos solucionar utilizando los repositorios y el material didáctico que dispuso el docente.

# Conclusión

Consideramos que el trabajo práctico cumple con su función, ya que nos generó afinidad en los temas trabajado en la materia,creando de esta manera un base sólida de conocimiento en Python. Las primeras funciones fueron fáciles, ya que utilizamos los temas vistos en la materia, pero mientras más avanzamos en los extras subía la dificultad, esto nos pareció que estaba perfecto, género de esta manera que nos metamos más a fondo en temas como por ejemplo django